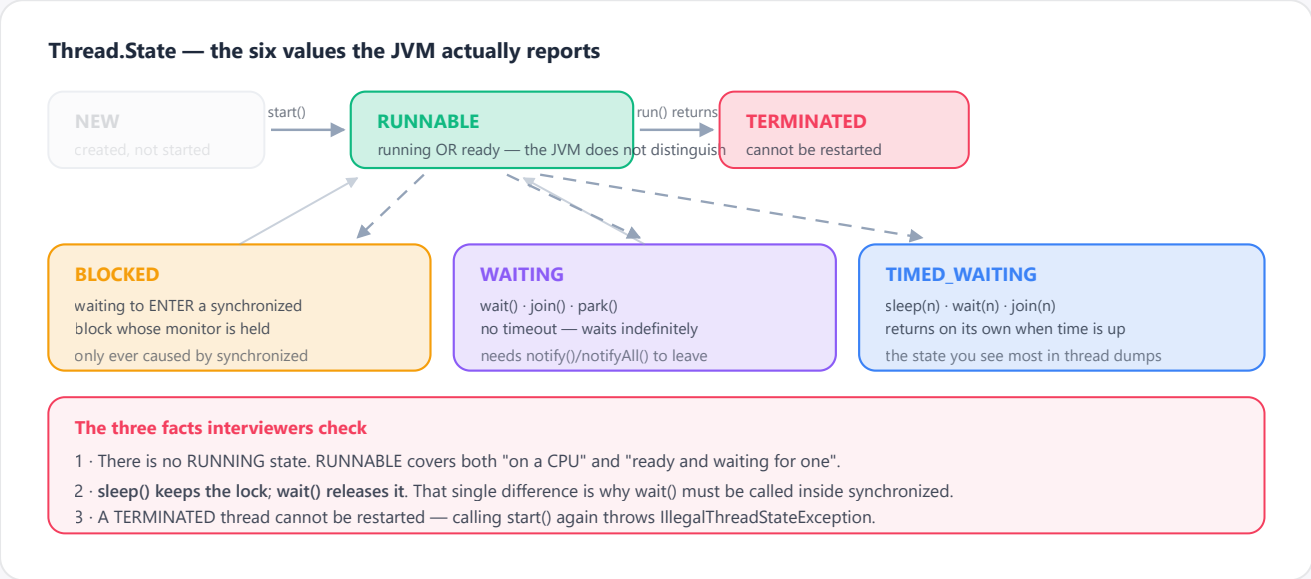
Threads: Creation and Lifecycle

Why it matters

The lifecycle is the vocabulary for every other concurrency conversation. When you read a thread dump at 3am, every line is a thread state — and knowing that `BLOCKED` means "waiting for a monitor" while `WAITING` means "someone must wake me" tells you immediately whether you are looking at lock contention or a missing `notify`. Interviewers also use the `sleep`-versus-`wait` question as a quick filter, because the answer hinges on one fact that separates people who have read about threads from people who have debugged them.

The concept, simply

A thread is an independent path of execution with its own stack (Chapter 1) sharing the heap with every other thread. `Thread.State` has exactly six values:



Note what is missing: there is no RUNNING state. RUNNABLE covers both actually executing and merely eligible to execute — the JVM does not distinguish, because that is the OS scheduler's business.

Two ways to define the work, and one is clearly better:

JAVA

Copy

```
// ✗ Extending Thread — burns your single inheritance slot on plumbing,
//   and conflates "what to do" with "how to run it".
class Worker extends Thread {
    public void run() { doWork(); }
}
new Worker().start();

// ✓ Implementing Runnable — the task is separate from the execution
//   mechanism, so the same object can go to a thread pool instead.
class Task implements Runnable {
    public void run() { doWork(); }
}
new Thread(new Task()).start();

// ✓ In practice, a lambda and an executor:
executor.submit(() -> doWork());
```

`start()` **versus** `run()` is the most-asked question here and has a crisp answer: `start()` asks the JVM to create a new OS thread which then calls `run()`. Calling `run()` directly is an ordinary method call on the current thread — no concurrency at all, and no error to tell you so.

`sleep()` **versus** `wait()` is the other. Both pause a thread, and they differ in one decisive way: `sleep()` **keeps every lock it holds;** `wait()` **releases the monitor it was called on.** That is why `wait()` must be called inside a `synchronized` block — it has to own the monitor in order to release it — and why sleeping while holding a lock is such an effective way to stall an application.

	`sleep(n)`	`wait()`
Declared on	`Thread` (static)	`Object`
Releases the lock	**No**	**Yes**

Needs `synchronized`	No	**Yes** — else `IllegalMonitorStateException`
Wakes on	Timeout	`notify()` / `notifyAll()`, or timeout
State	`TIMED_WAITING`	`WAITING` or `TIMED_WAITING`

Code examples

Cooperative cancellation — the only correct way to stop a thread:

JAVA

Copy

```java
class Worker implements Runnable {
    public void run() {
        // ✓ Check the interrupt flag; it is the standard cancellation sig
        while (!Thread.currentThread().isInterrupted()) {
            try {
                doUnitOfWork();
                Thread.sleep(100);
            } catch (InterruptedException e) {
                // ⚠ Catching it CLEARS the flag. Restore it, or the loop
                //   condition above will never see the cancellation.
                Thread.currentThread().interrupt();
                break;
            }
        }
        cleanup();
    }
}

Thread t = new Thread(new Worker());
t.start();
t.interrupt();      // ✓ polite request — the thread decides how to stop
t.join();           // wait for it to finish

// t.stop() is DEPRECATED and removed. It killed a thread mid-operation,
// releasing its locks while shared state was half-updated — corruption
// with no way to detect it.
```

Daemon threads, and the trap:

JAVA

```java
Thread t = new Thread(this::backgroundPoll);
t.setDaemon(true);          // must be set BEFORE start()
t.start();
// The JVM exits when only daemon threads remain — they do NOT keep it ali
// and they get no chance to finish. Never use a daemon thread for work th
// must complete, such as flushing a buffer or committing a transaction.
```

Reading a thread dump, which is what the states are actually for:

BASH

```bash
jcmd <pid> Thread.print       # or jstack <pid>
```

TEXT

```
"http-nio-8080-exec-3" #34 waiting for monitor entry [0x00007f ... ]
   java.lang.Thread.State: BLOCKED (on object monitor)
        at com.app.Cache.get(Cache.java:42)
        - waiting to lock <0x000000076b2f1234> (a com.app.Cache)
        - locked <0x000000076b2f9999> (a com.app.Registry)

// BLOCKED + "waiting to lock" = lock contention. Many threads BLOCKED on
// the same monitor address is a bottleneck; two threads each holding what
// the other wants is a deadlock (later in this chapter).
```

Under the hood

A Java thread is an OS thread. HotSpot maps each `java.lang.Thread` one-to-one onto a native thread, which is why they are expensive: roughly 1 MB of reserved stack each (`-Xss`, Chapter 1) plus kernel bookkeeping, and every context switch goes through

Virtual threads (Java 21) change this and are worth naming. A virtual thread is scheduled by the JVM onto a small pool of carrier threads, with its stack on the heap and grown on demand, so millions are feasible and blocking one is cheap. Crucially the API is unchanged — it is still `Thread`, still `Runnable` — so the whole "avoid blocking, go reactive" contortion becomes unnecessary for I/O-bound work. Chapter 10 covers it properly; here it is enough to know that the expensive-thread assumption behind pools is what it removes.

Thread priorities are close to meaningless. `setPriority` is a hint the OS may ignore entirely, and behaviour differs across platforms. Any design that depends on priority for correctness is broken. The same applies to `Thread.yield()`, which is a suggestion with no guarantee.

Uncaught exceptions kill only their own thread, silently by default — the stack trace goes to stderr and the application carries on with one fewer worker. Set an `UncaughtExceptionHandler`, or use an executor and inspect the returned `Future`, which captures the exception instead of losing it.

Common mistakes

- **Calling `run()` instead of `start()`.** No new thread, no error, no concurrency.
- **Swallowing `InterruptedException`.** Catching it clears the flag; restore it or rethrow, or cancellation stops working everywhere above you.
- **Using `Thread.stop()`.** Deprecated and removed — it corrupted state by design.
- **Extending `Thread`.** Wastes the inheritance slot and couples the task to its execution.
- **`sleep()` while holding a lock.** Every other thread waits for the whole duration.
- **Calling `wait()` outside `synchronized`.** `IllegalMonitorStateException`.
- **Restarting a terminated thread.** `IllegalThreadStateException` — threads are single-use.
- **Relying on priorities.** A hint the OS is free to ignore.
- **Creating threads per request.** Roughly 1 MB each — use a pool, or virtual threads.

Q: "Difference between `start()` and `run()`?"

A: `start()` asks the JVM to create a new OS thread, which then invokes `run()`. Calling `run()` yourself is a plain method call on the current thread — the code executes, but sequentially, with no concurrency and no warning that you got it wrong. Calling `start()` twice throws `IllegalThreadStateException`.

Q: "`sleep()` vs `wait()`?"

A: The decisive difference is locks: `sleep()` holds every lock it has, `wait()` releases the monitor it was called on. That is also why `wait()` must be inside `synchronized` — it must own the monitor to release it — and why it needs `notify()` to resume, whereas `sleep()` wakes on its own. `sleep` is static on `Thread`; `wait` is on `Object`.

Q: "What are the thread states?"

A: `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, `TERMINATED`. The notable detail is that there is no `RUNNING` — `RUNNABLE` covers both executing and ready-to-execute, because scheduling is the OS's job. And `BLOCKED` specifically means waiting to enter a `synchronized` block; waiting on a `Lock` shows as `WAITING`.

Q: "How do you stop a thread?"

A: Cooperatively. Call `interrupt()`, and have the thread check `isInterrupted()` in its loop and handle `InterruptedException` from blocking calls by cleaning up and exiting. The critical detail is that catching `InterruptedException` clears the flag, so you must restore it with `Thread.currentThread().interrupt()` or rethrow. `Thread.stop()` was removed because it released locks mid-update and left shared state corrupted.

A: One that does not keep the JVM alive — when only daemon threads remain, the JVM exits and they are abandoned wherever they are. It must be marked with `setDaemon(true)` before `start()`. Good for background housekeeping like metrics polling; wrong for anything that must complete, since it gets no chance to finish.

Q: "What happens to an uncaught exception in a thread?"
A: It terminates that thread only; the rest of the application continues, and by default the trace just goes to stderr — so a worker can quietly disappear. Install an `UncaughtExceptionHandler`, or submit tasks to an executor, where the exception is captured in the `Future` and surfaces when you call `get()`.

Q: "Why are threads expensive?"
A: Each maps one-to-one onto an OS thread, reserving around a megabyte of stack plus kernel structures, and every switch involves the scheduler. That is why pools exist and why servers cap concurrency well below the number of connections. Java 21's virtual threads change the calculus — they are JVM-scheduled with heap-allocated stacks, so millions are practical and blocking is cheap.

Cheat-card

- Six states: `NEW` · `RUNNABLE` · `BLOCKED` · `WAITING` · `TIMED_WAITING` · `TERMINATED`. **No RUNNING.**
- `start()` = new thread. `run()` = ordinary method call.
- `sleep` **keeps locks** · `wait` **releases the monitor** (and needs `synchronized`).
- `BLOCKED` = waiting for a **monitor**; `Lock` waiting shows as `WAITING`.
- Stop threads **cooperatively** via `interrupt()`; catching `InterruptedException` **clears the flag** — restore it.

- ~1 MB stack per thread → pools. Java 21 **virtual threads** remove that constraint.
- Uncaught exception kills one thread **silently** — set a handler or use a `Future`.

QUICK CHECK — no marks, just for you.

1. What is the difference between calling start() and calling run() on a Thread?

- ◯ run() creates the thread and start() schedules it
- ◯ start() is deprecated in favour of run()
- ◯ start() creates a new OS thread which then invokes run(); calling run() directly is an ordinary method call on the current thread with no concurrency at all
- ◯ They are equivalent

Check answer

2. What is the decisive difference between Thread.sleep() and Object.wait()?

- ◯ sleep releases locks and wait keeps them
- ◯ sleep is more accurate
- ◯ wait can only be used on collections
- ◯ sleep KEEPS every lock it holds; wait RELEASES the monitor it was called on — which is why wait must be called inside synchronized

Check answer

3. You catch InterruptedException and log it. What have you broken?

Catching it CLEARS the interrupt flag, so code above you that relies on the interrupt to stop work never sees it — cancellation and shutdown silently stop functioning

- ○ The thread terminates immediately
- ○ The exception is rethrown automatically at the next blocking call
- ○ Nothing, as long as you log it

Check answer

‹ Previous
Optional Done Right

Next ›
Race Conditions and synchronized

LEARN MORE

ARTICLES & DOCS

- Thread and Thread.State — API docs
  docs.oracle.com
- Concurrency — The Java Tutorials
  docs.oracle.com

VIDEOS



Java Multithreading: Synchronization, Locks, Executors, Deadlock, CountdownLatch & CompletableFuture
Engineering Digest

Hand-picked references for this lesson.